C++ PROGRAMMING

Assignment – 01

Logic Building & Fundamental Programming Concepts

Course: Programming in C++
Difficulty Level: Intermediate
Total Questions: 20
Topics Covered: Variables, Operators, Conditional Statements, Loops, Functions, Strings and Basic Arrays.

GENERAL INSTRUCTIONS

1. Write a separate C++ program for each question.
2. Use appropriate data types and meaningful variable names.
3. Use conditional statements and loops wherever required.
4. Divide complex programs into smaller functions wherever applicable.
5. Handle invalid inputs and special cases appropriately.
6. Do not use advanced STL containers, sorting algorithms or other DSA concepts.
7. Write clean, properly indented and readable code.
8. Test each program with multiple test cases.

SECTION A

VARIABLES, OPERATORS & CONDITIONAL STATEMENTS

Question 01 – Electricity Bill Calculator

Write a C++ program to calculate the electricity bill based on the total number of units consumed.

The electricity charges are calculated according to the following slabs:

Units Consumed	Rate per Unit

First 100 units	₹2
Next 100 units	₹3
Next 200 units	₹5
Above 400 units	₹7

An additional fixed charge of ₹50 must be added to the final bill.

Requirements:

- Accept the total units consumed as input.
- Calculate the bill using progressive slab-based taxation.
- Display the final electricity bill.

Sample Input:

```
None
250
```

Sample Output:

```
None
Total Electricity Bill: Rs. 850
```

Question 02 – Triangle Classification

Write a C++ program that accepts three angles of a triangle and determines whether they form a valid triangle.

If the triangle is valid, classify it into one of the following categories:

1. Acute-angled triangle.
2. Right-angled triangle.
3. Obtuse-angled triangle.

Requirements:

- A valid triangle must have all positive angles.
- The sum of all three angles must be exactly 180 degrees.
- Display an appropriate message for invalid input.

Sample Input:

```
None
60 60 60
```

Sample Output:

```
None
Valid Triangle
Type: Acute Angled Triangle
```

Question 03 – Date Validation

Write a C++ program to check whether a given date is valid.

The program should accept the day, month and year as input.

Requirements:

- Validate the month and day.
- Consider different numbers of days in different months.
- Handle leap years correctly.
- February must have 29 days in a leap year and 28 days otherwise.

Sample Input:

```
None
29 2 2024
```

Sample Output:

```
None
Valid Date
```

Additional Test Case:

```
None
Input: 29 2 2023
Output: Invalid Date
```

Question 04 – Income Tax Calculator

Write a C++ program to calculate the income tax payable based on the annual income of an individual.

Use the following progressive tax slabs:

Annual Income	Tax Rate
Up to ₹3,00,000	0%
₹3,00,001 – ₹7,00,000	5%
₹7,00,001 – ₹10,00,000	10%
₹10,00,001 – ₹12,00,000	15%
Above ₹12,00,000	20%

Requirements:

- Accept annual income as input.
- Apply tax progressively to each income slab.
- Calculate the total tax payable.
- Display the final tax amount.

Sample Input:

```
None
800000
```

Sample Output:

```
None
Total Tax: Rs. 30000
```

Question 05 – Menu-Based Calculator

Write a C++ program to implement a simple calculator using the `switch` statement.

The calculator must support the following operations:

- Addition (+)
- Subtraction (-)
- Multiplication (*)
- Division (/)

- Modulus (%)

Requirements:

- Accept two numbers and an operator from the user.
- Perform the selected arithmetic operation.
- Handle division by zero.
- Handle invalid operators.
- Use appropriate data types to support decimal calculations wherever applicable.

Sample Input:

```
None
Enter first number: 25
Enter operator: *
Enter second number: 4
```

Sample Output:

```
None
Result: 100
```

SECTION B

LOOPS & NUMBER-BASED LOGIC

Question 06 – Number Guessing Game

Write a C++ program to implement a simple number guessing game.

The program should contain a predefined secret number. The user must guess the number within a maximum of seven attempts.

Requirements:

- If the guessed number is greater than the secret number, display "Too High".
- If the guessed number is smaller than the secret number, display "Too Low".
- If the guessed number matches the secret number, display "Correct Guess".
- If the user fails within seven attempts, display the correct number.
- The program must stop immediately when the correct number is guessed.

Question 07 – Digit Frequency Counter

Write a C++ program that accepts an integer and calculates the frequency of each digit from 0 to 9.

Requirements:

- Count the occurrences of every digit.
- Display the frequency of all ten digits, including digits that do not appear.
- Do not convert the integer into a string.
- Use arithmetic operations and loops to extract individual digits.

Sample Input:

```
None
12233345
```

Sample Output:

```
None
Digit Frequency

0 : 0
1 : 1
2 : 2
3 : 3
4 : 1
5 : 1
6 : 0
7 : 0
8 : 0
9 : 0
```

Question 08 – Strong Number Checker

Write a C++ program to check whether a given number is a Strong Number.

A Strong Number is a number whose value is equal to the sum of the factorials of its individual digits.

For example:

145 = 1! + 4! + 5!

145 = 1 + 24 + 120

145 = 145

Requirements:

- Accept an integer as input.
- Extract individual digits using loops.
- Calculate the factorial of each digit.
- Calculate the sum of all digit factorials.
- Determine whether the number is a Strong Number.

Sample Input:

```
None
145
```

Sample Output:

```
None
Strong Number
```

Question 09 – Multiplication Table Challenge

Write a C++ program to print multiplication tables from 1 to N.

Requirements:

- Accept N from the user.
- Use nested loops to generate the tables.
- Skip all multiplication results that are multiples of 5.
- Stop printing the current table when its product exceeds 50.
- Continue with the next multiplication table.

Sample Input:

```
None
3
```

Sample Output:

```
None
Table of 1:
```

```
1 x 1 = 1
1 x 2 = 2
1 x 3 = 3
1 x 4 = 4
1 x 6 = 6
...

Table of 2:
2 x 1 = 2
2 x 2 = 4
2 x 3 = 6
...
```

Question 10 – Consecutive Number Pattern

Write a C++ program to print the following number pattern for a given value of N.

Sample Input:

```
None
5
```

Sample Output:

```
None
1
2 3
4 5 6
7 8 9 10
11 12 13 14 15
```

Requirements:

- Use nested loops.
- Maintain a separate counter to generate consecutive numbers.
- The number of elements in each row must increase by one.

SECTION C

FUNCTIONS & MODULAR PROGRAMMING

Question 11 – Number Property Checker

Write a C++ program to check different mathematical properties of a given integer using separate user-defined functions.

Create functions to check whether the number is:

1. Even or Odd.
2. Prime or Composite.
3. Palindrome.
4. Armstrong Number.

Requirements:

- Each property must be checked using a separate function.
- The main function should call all the required functions.
- Display the result of each property separately.

Sample Input:

```
None
153
```

Sample Output:

```
None
Even/Odd: Odd
Prime/Composite: Composite
Palindrome: No
Armstrong: Yes
```

Question 12 – Custom Power Function

Write a C++ program to calculate the power of a number using a user-defined function.

Create the following function:

```
None
double power(double base, int exponent);
```

Requirements:

- Do not use the built-in `pow()` function.
- Handle positive exponents.
- Handle zero as an exponent.
- Handle negative exponents.
- Return the calculated result.

Sample Input:

```
None
Base: 2
Exponent: 5
```

Sample Output:

```
None
Result: 32
```

Question 13 – Menu-Driven Number Operations

Write a menu-driven C++ program that performs different operations on an integer.

Display the following menu:

```
None
========== NUMBER OPERATIONS ==========

1. Find Factorial
2. Check Prime
3. Reverse Number
4. Find Sum of Digits
5. Check Palindrome
6. Exit

Enter your choice:
```

Requirements:

- Implement every operation using a separate user-defined function.
- Use a loop to repeatedly display the menu.
- Execute the selected operation.
- Continue until the user selects Exit.
- Handle invalid menu choices.

Question 14 – Function-Based Pattern Generator

Write a C++ program that prints different patterns using separate user-defined functions.

The program must support the following patterns:

Pattern 1: Increasing Triangle

```
None
*
**
***
****
*****
```

Pattern 2: Decreasing Triangle

```
None
*****
****
***
**
*
```

Pattern 3: Pyramid Pattern

```
None
    *
   ***
  *****
 *******
*********
```

Requirements:

- Implement each pattern using a separate function.
- Accept the number of rows from the user.
- Allow the user to select which pattern to print.
- Use loops to generate the patterns.

SECTION D

STRING MANIPULATION

Question 15 – Character Type Counter

Write a C++ program that accepts a string and counts the occurrences of different character types.

The program must count:

1. Uppercase letters.
2. Lowercase letters.
3. Numeric digits.
4. White spaces.
5. Special characters.

Sample Input:

```
None
Hello World 123!
```

Sample Output:

```
None
Uppercase Letters: 2
Lowercase Letters: 8
Digits: 3
Spaces: 2
Special Characters: 1
```

Question 16 – Word Counter

Write a C++ program to count the total number of words in a given sentence.

Requirements:

- Accept a complete sentence containing spaces.
- Handle multiple consecutive spaces.
- Ignore leading and trailing spaces.
- Count each valid word exactly once.
- Do not use `stringstream`.

Sample Input:

```
None
   C++   is   an interesting language
```

Sample Output:

```
None
Total Words: 5
```

Question 17 – String Case Converter

Write a C++ program to convert the case of all alphabetic characters in a given string.

Requirements:

- Convert lowercase letters into uppercase.
- Convert uppercase letters into lowercase.
- Keep numeric digits unchanged.
- Keep special characters and spaces unchanged.
- Do not use built-in case-conversion functions such as `toupper()` or `tolower()`.

Sample Input:

```
None
Hello World 123!
```

Sample Output:

```
None
hELLO wORLD 123!
```

SECTION E

BASIC ARRAY OPERATIONS

Question 18 – Array Statistics

Write a C++ program that accepts N integer elements into an array and calculates different statistical properties.

The program must calculate:

1. Sum of all elements.
2. Average of all elements.
3. Maximum element.
4. Minimum element.
5. Total number of even elements.
6. Total number of odd elements.

Sample Input:

```
None
N = 6

Array:
10 15 20 25 30 35
```

Sample Output:

```
None
Sum: 135
Average: 22.5
Maximum: 35
Minimum: 10
Even Elements: 3
Odd Elements: 3
```

Question 19 – Array Comparison

Write a C++ program that accepts two arrays of equal size and compares them.

The program must determine:

1. Whether both arrays are identical.
2. Whether corresponding elements are equal at every position.
3. Whether the sum of both arrays is equal.

Requirements:

- Accept two arrays of equal size.
- Compare their corresponding elements.
- Calculate the sum of each array.
- Display the results separately.
- Use basic loops and conditional statements.

Sample Input:

```
None
Array 1:
1 2 3 4 5

Array 2:
1 2 3 4 5
```

Sample Output:

```
None
Arrays are Identical.
Corresponding Elements are Equal.
Both Array Sums are Equal.
```

Question 20 – Array Element Frequency

Write a C++ program to find the frequency of each distinct element in an array.

Requirements:

- Accept N integer elements.
- Count the frequency of every distinct element.
- Display each distinct element only once.
- Use nested loops.
- You may use an additional visited array to avoid printing duplicate elements.

Sample Input:

```
None
N = 8

Array:
1 2 2 3 3 3 4 4
```

Sample Output:

```
None
Element Frequency

1 : 1
2 : 2
3 : 3
4 : 2
```

END OF ASSIGNMENT

Submission Guidelines

- Submit all 20 programs separately.
- Include the question number and question title in every source file.
- Ensure that every program compiles and executes successfully.
- Include appropriate comments wherever necessary.
- Test programs using different input values and edge cases.

Learning Objective: Develop strong logical thinking and problem-solving abilities through variables, operators, conditional statements, loops, functions, strings and basic arrays, building a solid foundation for advanced C++ programming.

C++ Intermediate Practice Set (20 Questions)

Intermediate
20 Problems
Coding Practice

Section 1: Basics, Variables & Operators

Q1. Digit Manipulation

Write a program that takes an integer and:

- Counts the total number of digits.
- Calculates the sum of its digits.
- Reverses the number.
- Checks whether it is a palindrome.

Do not convert the number into a string.

Q2. Number Transformation

Given an integer `n`, replace every digit:

- `0` with `5`
- `5` with `0`
- All other digits remain unchanged.

Example:

```
None
Input:  150205
Output: 010250
```

Consider how you will preserve leading zeroes in the output.

Q3. Operator Challenge

Without using `pow()`, calculate:

$$1^1 + 2^2 + 3^3 + \cdots + n^n$$

Example:

```
None
Input: 4
Output: 288
```

Q4. Armstrong Number Range

Print all Armstrong numbers between two given integers.

Example:

```
None
Input: 100 1000
Output: 153 370 371 407
```

Your solution should work for numbers with different digit counts.

Section 2: Conditional Statements & Loops

Q5. Prime Number Analysis

Given an integer `n`:

- Check whether it is prime.
- Find its smallest prime factor.
- Print all its factors.

Example:

```
None
Input: 36
Prime: No
Smallest Prime Factor: 2
Factors: 1 2 3 4 6 9 12 18 36
```

Q6. Pattern Without Nested Loops

Print the following pattern for a given `n`:

For `n = 5`:

```
None
1
```

```
22
333
4444
55555
```

Try solving it using only one loop (no nested loops).

Q7. GCD and LCM

Take two integers and calculate their GCD and LCM.

Constraints:

- Do not use any built-in GCD function.
- Implement GCD using the Euclidean algorithm.
- Avoid multiplication overflow where possible.

Q8. Decimal to Binary

Convert a decimal integer into binary without using arrays, strings, or built-in conversion functions.

Example:

```
None
Input: 25
Output: 11001
```

Section 3: Functions

Q9. Function Overloading

Create overloaded functions named `calculate()`:

- Two integers → return their sum.
- Three integers → return their product.
- Two doubles → return their average.

Demonstrate all three functions in `main()`.

Q10. Recursive Digit Sum

Write a recursive function that calculates the sum of digits of a given integer.

Example:

```
None
Input: 98765
Output: 35
```

Bonus: Repeat the process until you get a single-digit number.

```
None
98765 → 35 → 8
```

Q11. Pass by Value vs Reference

Write a program with two functions:

- `swapValue(int a, int b)`
- `swapReference(int &a, int &b)`

Demonstrate why one function modifies the original variables while the other does not.

Q12. Function-Based Number Properties

Create separate functions to check whether a number is:

- Prime
- Perfect
- Strong
- Palindrome

Take a number from the user and display all applicable properties.

Example:

```
None
Input: 6
Prime: No
Perfect: Yes
Strong: No
Palindrome: Yes
```

Section 4: Strings

Q13. Character Frequency

Given a string, count the frequency of every character.

Example:

```
None
Input: programming

Output:
p : 1
r : 2
o : 1
g : 2
a : 1
m : 2
i : 1
n : 1
```

Do not use `map` or `unordered_map`.

Q14. String Palindrome (Advanced)

Check whether a string is a palindrome.

Conditions:

- Ignore spaces.
- Ignore uppercase/lowercase differences.
- Ignore punctuation.

Example:

```
None
Input: A man, a plan, a canal: Panama
Output: Palindrome
```

Q15. Longest Word in a Sentence

Given a sentence, find:

- The longest word.
- Its length.
- Its starting position in the original string.

Example:

```
None
Input: C++ programming is interesting

Longest Word: programming
Length: 11
Starting Position: 4
```

Do not use `stringstream`.

Q16. String Compression

Compress a string by replacing consecutive repeated characters with the character followed by its frequency.

Example:

```
None
Input: aaabbccccdaa

Output: a3b2c4d1a2
```

Do not use additional arrays or maps.

Section 5: Arrays

Q17. Second Largest Distinct Element

Find the second-largest distinct element in an array.

Example:

```
None
Input: 10 20 20 8 15 10

Output: 15
```

Conditions:

- Do not sort the array.
- Do not use another array.
- Try solving in a single traversal.

Q18. Array Rotation

Rotate an array to the right by k positions.

Example:

```
None
Input:
n = 7
k = 3
Array = 1 2 3 4 5 6 7

Output:
5 6 7 1 2 3 4
```

Bonus: Solve it in-place using the reversal algorithm.

Q19. Maximum Subarray Sum

Find the contiguous subarray with the maximum sum.

Example:

```
None
Input:
-2 1 -3 4 -1 2 1 -5 4

Output:
6
Subarray: 4 -1 2 1
```

Try implementing Kadane's algorithm.

Q20. Array Equilibrium Index

Find an index where the sum of all elements on the left equals the sum of all elements on the right.

Example:

```
None
Input:
1 3 5 2 2

Output:
```

```
2
```

Explanation:

```
None
Left Sum  = 1 + 3 = 4
Right Sum = 2 + 2 = 4
```

Use 0-based indexing.